SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

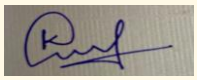
Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 2 – Agile Sprint Planning & User Story Workshop

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 2 – **Agile Sprint Planning & User Story Workshop**
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	Kosicram K	**Reg. No.:**	192571079
Date:		**Duration:**	60 minutes

Code of Conduct

I Kosicram K / 192571079 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Annexure A – Agile Sprint Planning & User Story Workshop

Instructions to Students:

Each student is assigned an individual software project problem statement. Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

1. **Identify the Project Vision and Stakeholders**
   - Define the project's vision, objectives, and identify all key stakeholders involved.
2. **Create Epics and User Stories**
   - Break down the requirements into Epics and formulate well-structured User Stories using the format: *As a <user>, I want <feature>, so that <benefit>.*
3. **Define Acceptance Criteria**
   - Write clear and measurable acceptance criteria for each selected user story using the Given–When–Then format or equivalent.
4. **Prioritize the Product Backlog**
   - Organize and prioritize the product backlog based on business value, customer needs, and project dependencies.
5. **Plan the Sprint Backlog**
   - Select high-priority user stories for the first sprint and divide them into actionable development tasks.
6. **Estimate Effort Using Story Points**
   - Assign story points to each selected user story using an appropriate estimation technique (e.g., Fibonacci sequence or Planning Poker) and justify your estimates.
7. **Present the Sprint Goal and Expected Deliverables**
   - Define a clear sprint goal and describe the expected sprint deliverables, including the features to be completed at the end of the sprint.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Project Vision & Stakeholder Identification (30%)	Project vision is clear, comprehensive, and aligned with the problem statement. All relevant stakeholders and their roles are correctly identified.	Vision is clear with minor omissions. Most stakeholders are correctly identified.	Vision is partially defined. Some stakeholders are missing or incorrectly identified.	Vision is unclear or incomplete. Stakeholders are poorly identified or missing.
Epics & User Stories (25%)	Epics are well-defined and user stories are complete, follow Agile format, and fully capture functional requirements.	Epics and user stories are mostly complete with minor formatting or requirement issues.	User stories are partially complete with limited Agile structure.	User stories are incomplete, unclear, or do not follow Agile practices.
Acceptance Criteria (20%)	Acceptance criteria are specific, measurable, testable, and aligned with all user stories.	Acceptance criteria are mostly clear and testable with minor gaps.	Acceptance criteria are partially defined and lack clarity or completeness.	Acceptance criteria are missing or not aligned with user stories.
Product Backlog Prioritization & Sprint Planning (15%)	Product backlog is logically prioritized, sprint backlog is realistic, and task breakdown is complete.	Backlog prioritization and sprint planning are mostly appropriate with minor issues.	Prioritization or sprint planning is partially complete with limited justification.	Backlog prioritization and sprint planning are poorly organized or incomplete.
Story Point Estimation, Sprint Goal & Presentation (10%)	Story point estimation is well justified, sprint goal is clear, deliverables are realistic, and presentation is professional and well organized.	Estimation and sprint goal are mostly appropriate with minor inconsistencies. Presentation is clear.	Estimation or sprint goal lacks justification. Presentation is adequate but incomplete.	Estimation is unrealistic or missing. Sprint goal, deliverables, and presentation are unclear or incomplete.

Criteria	Mark
Project Vision & Stakeholder Identification **(30%)**	/5
Epics & User Stories **(25%)**	/5
Acceptance Criteria **(20%)**	/5
Product Backlog Prioritization & Sprint Planning **(15%)**	/5
Story Point Estimation, Sprint Goal & Presentation **(10%)**	/5
TOTAL	/25

AI-Powered Phishing Detection and Cyber Threat Prevention System

Introduction

Cybersecurity is an essential part of modern software engineering because individuals and organizations increasingly depend on digital platforms for communication, banking, online transactions, and data storage. As internet usage continues to grow, cybercriminals use sophisticated techniques to steal sensitive information, gain unauthorized access to systems, and deceive users through fraudulent emails, messages, and websites. Phishing is one of the most common cyber threats, in which attackers impersonate trusted individuals or organizations to trick users into revealing confidential information.

The proposed AI-Powered Phishing Detection and Cyber Threat Prevention System uses artificial intelligence, machine learning, URL analysis, and natural language processing to detect phishing attempts.

Understand the Problem Statement

Phishing attacks exploit human trust and weaknesses in digital communication. Attackers frequently create fraudulent websites that resemble legitimate banking portals, social media platforms, educational websites, and organizational login pages. They distribute links through emails, text messages, and online advertisements to convince users to disclose passwords, financial information, or other confidential data.

Traditional security systems identify suspicious content using known malicious URLs, predefined rules, and previously reported attack patterns. However, attackers can create new domains, modify website addresses, change email wording, and use misleading links to bypass these methods. Consequently, users may receive fraudulent messages that appear genuine and are not detected by basic filtering mechanisms.

Another challenge is that users may not understand the warning signs of phishing. Messages containing urgent requests, suspicious attachments, misleading sender information, or unusual login links can appear convincing. Without appropriate detection and awareness mechanisms, users may unknowingly provide sensitive information to attackers.

The main problem is the absence of an integrated and intelligent platform that can analyze different phishing indicators, estimate the risk associated with suspicious content, and provide understandable warnings before users interact with potentially dangerous links or messages.

The proposed system addresses this problem by using machine learning models trained on suitable phishing and legitimate samples. URL analysis examines features such as URL length, unusual characters, suspicious patterns, and domain-related information. Natural language processing can analyze email subject lines and message content for suspicious language, impersonation, and social engineering indicators. A rule-based verification component can supplement the model by checking additional security indicators.

After analysis, the system classifies the submitted content as potentially legitimate, suspicious, or likely phishing. It generates a risk score and explains the indicators that influenced the result. Security administrators can review detected threats, examine incident records, and generate

reports. The system should clearly communicate that automated predictions are risk assessments rather than guarantees of safety.

Objectives of the Case Study

- **Understand how phishing attacks exploit weaknesses in digital communication and user awareness.**
- **Identify the limitations of traditional phishing detection techniques.**
- **Study the application of artificial intelligence and machine learning in cybersecurity.**
- **Develop a system for analyzing suspicious URLs, emails, and message content.**

Scope of the Case Study

The scope includes phishing URL detection, suspicious email analysis, text classification, risk scoring, alert generation, incident management, user authentication, and security reporting. The system can be implemented as a web application that accepts URLs or email text for analysis and displays the results through a user-friendly interface.

Identify Key Stakeholders and Challenges

A phishing detection system involves several stakeholders who contribute to its development, operation, and evaluation. Each stakeholder has different responsibilities, and effective coordination is required to ensure that the system remains reliable, secure, and useful.

Key Stakeholders

- **End Users: Individuals who submit suspicious URLs or email content for analysis. They depend on understandable warnings to make safer decisions.**
- **Cybersecurity Analysts: Security professionals who examine detected threats, investigate suspicious activities, and validate system-generated classifications.**
- **System Administrators: Responsible for managing user accounts, application configurations, access permissions, system availability, and maintenance.**
- **Software Developers: Design, implement, test, and maintain the web application, detection algorithms, and supporting services.**
- **Legal and Compliance Teams: Ensure that data collection, storage, processing, and incident handling follow applicable privacy and security requirements.**
- **Threat Intelligence Providers: Supply domain reputation information, phishing indicators, and other relevant threat intelligence when suitable integrations are available.**

Major Challenges

- **Newly created phishing websites may not appear in existing blocklists.**
- **Attackers frequently modify URLs, email wording, and website designs to avoid detection.**
- **Machine learning models may incorrectly classify legitimate content as malicious or fail to identify some phishing attempts.**
- **High-quality, representative, and regularly updated training datasets are required.**

Analyze the Current Approach

Traditional phishing detection systems use several methods, including blocklists, predefined rules, sender verification, spam filtering, and signature-based detection. These techniques identify threats

by comparing incoming content with known malicious indicators or checking whether messages violate established security rules.

Although these methods are useful, their effectiveness depends on the quality and freshness of threat information. A newly registered phishing domain may not yet be included in a blocklist. Similarly, a fraudulent email that uses legitimate-looking language and a previously unseen URL may pass through basic filters.

Machine learning can improve this process by identifying patterns associated with phishing rather than relying exclusively on previously reported URLs. When combined with traditional security checks, it can provide an additional method for identifying suspicious content.

Strengths of the Existing Approach

- **Blocklists can quickly identify previously reported malicious URLs.**
- **Rule-based filters are relatively simple to implement and maintain.**
- **Sender authentication mechanisms help identify certain types of email spoofing.**

Limitations of the Existing Approach

- **Newly created phishing websites may not be present in existing blocklists.**
- **Fixed rules may not detect attacks that use modified wording or unfamiliar URL patterns.**
- **Legitimate websites can occasionally be flagged incorrectly.**
- **Separate security tools may not provide a unified view of phishing incidents.**
- **Users may ignore warnings if the reasons for the warning are unclear.**

Problems and Gaps

The primary gap is the limited ability of traditional systems to adapt to unfamiliar phishing patterns while providing a centralized view of detected threats. Blocklists are useful for recognizing known malicious domains, but they cannot independently identify every newly created phishing website.

Another gap is the lack of understandable explanations for users. A warning that simply labels a website as suspicious may not help the user understand the reason for the classification. The proposed system addresses this issue by presenting relevant indicators, such as unusual URL structure, suspicious language, misleading domain patterns, or available domain reputation information.

Propose a Suitable Solution Using Software Engineering Principles

A suitable solution is to develop an AI-Powered Phishing Detection and Cyber Threat Prevention System that combines machine learning, rule-based verification, secure data handling, and centralized threat monitoring.

The system accepts a URL or email text from the user and extracts relevant features for analysis. The URL analysis component examines structural characteristics, while the email analysis component processes message content to identify suspicious patterns. A trained machine learning model evaluates the extracted features and estimates whether the submitted content resembles known phishing or legitimate examples.

The detection engine combines the model's prediction with available verification rules and threat intelligence, where applicable. It then produces a classification, risk score, and explanation of important warning indicators. The results are displayed through a web interface, allowing users to understand the potential risk before proceeding.

The proposed system should initially operate as a decision-support tool. Automatic blocking or quarantine can be added through controlled integrations after the system has been thoroughly tested.

Main Functional Modules

- **User Authentication Module: Provides secure login, account management, and role-based access to system functions.**
- **URL Analysis Module: Extracts URL features such as length, special characters, suspicious tokens, subdomain patterns, and other relevant structural indicators.**
- **Email Analysis Module: Examines email text, subject lines, suspicious requests, misleading claims, and social engineering indicators.**
- **Feature Extraction Module: Converts URLs and message content into structured information suitable for classification.**
- **Machine Learning Detection Module: Uses a trained classification model to identify patterns associated with phishing and legitimate content.**
- **Rule-Based Verification Module: Applies additional checks to identify suspicious characteristics and complement machine learning predictions.**
- **Threat Intelligence Module: Uses available domain reputation services or trusted threat feeds to obtain additional indicators when configured.**
- **Risk Scoring and Explanation Module: Combines available detection results into a risk assessment and presents the main factors contributing to the decision.**
- **Alert and Notification Module: Generates warnings when submitted content is classified as suspicious or high risk.**
- **Incident Management Module: Records reported threats, investigation status, analyst decisions, and resolution details.**
- **Dashboard and Reporting Module: Displays detection statistics, threat categories, incident history, and system performance reports.**
- **Audit and Administration Module: Records important administrative actions and supports monitoring, configuration, and maintenance.**

Functional Requirements

- **The system shall allow authorized users to submit URLs and email text for analysis.**
- **The system shall extract relevant features from submitted content.**
- **The system shall classify submitted content using a trained detection model and supporting rules.**
- **The system shall generate a risk score or risk category for each analysis.**
- **The system shall display the main indicators that contributed to the classification.**
- **The system shall alert users when content is identified as potentially suspicious.**
- **The system shall maintain detection history according to the configured data retention policy.**
- **The system shall allow authorized security analysts to review and update incident status.**
- **The system shall provide dashboards and reports showing detected threats and classification statistics.**

- **The system shall support role-based access control for users and administrators.**
- **The system shall allow authorized administrators to update detection rules and deploy improved models.**

Non-Functional Requirements

- **Security: The application must protect accounts, submitted content, stored records, and administrative functions through appropriate security controls.**
- **Reliability: The system should provide consistent analysis results and handle failures safely.**
- **Performance: The application should analyze ordinary submissions within a reasonable response time.**
- **Scalability: The architecture should support increasing numbers of users and analysis requests.**
- **Maintainability: Separate modules should allow detection algorithms and application components to be updated independently.**
- **Usability: The interface should present risk assessments and warnings in clear, understandable language.**
- **Privacy: Sensitive submitted content should be minimized, protected, and retained only as necessary.**
- **Accuracy: Model performance should be evaluated using suitable metrics, including precision, recall, F1-score, and false-positive rate.**
- **Availability: The application should handle temporary service interruptions and recover without unnecessary data loss.**

System Architecture (Block Diagram)

SYSTEM USERS
End Users | Cybersecurity Analysts | System Administrators
↓
WEB INTERFACE
Login | URL Submission | Email Analysis | Risk Results | Dashboard | Reports
↓
AI-POWERED PHISHING DETECTION SYSTEM
Input Validation | Feature Extraction | Machine Learning Detection | Rule-Based Verification | Risk Scoring
↓
CYBERSECURITY SERVICES
URL Analysis | Email Text Analysis | Threat Intelligence | Alert Generation | Incident Management
↓
EXTERNAL AND INTERNAL DATA SOURCES
Training Dataset | Threat Intelligence Feeds | Domain Reputation Information | Detection Rules
↓

SECURITY DATABASE
User Records | Detection Results | Incident History | Audit Logs | Model and Rule Versions
↓
ADMINISTRATIVE CONTROLS
Access Management | Data Protection | Model Evaluation | Monitoring | Backup and Recovery

The architecture follows a modular, layered approach. The web interface allows users to submit suspicious content and view the analysis results. The application layer validates inputs and coordinates the detection process. The AI detection engine extracts features, applies the trained model, and combines its output with relevant security checks. The threat intelligence component retrieves additional information when external services are configured and available.

The database stores essential application records, detection results, incident details, and audit information. Access controls protect administrative functions and sensitive records. Model training and evaluation should be performed separately from ordinary user requests, with validated model versions deployed to the detection service.

Software Engineering Principles Used

- **Modularity: Separate components handle input validation, feature extraction, detection, reporting, and incident management.**
- **Maintainability: Individual modules and machine learning models can be updated without redesigning the entire application.**
- **Reliability: Input validation, error handling, testing, and controlled recovery improve system stability.**
- **Security by Design: Authentication, authorization, data protection, and secure configuration are considered throughout development.**
- **Least Privilege: Users and services receive only the permissions required to perform their functions.**
- **Separation of Concerns: The web interface, detection engine, data storage, and administrative services have distinct responsibilities.**
- **Testability: Detection algorithms and application modules can be tested independently and together.**
- **Privacy by Design: The system limits unnecessary data collection and protects stored information.**

Justify the Chosen Methodology/Framework

Agile methodology is suitable for developing the AI-Powered Phishing Detection and Cyber Threat Prevention System because cybersecurity threats, detection requirements, and machine learning techniques change continuously. Agile development divides the project into manageable iterations, allowing the team to build, test, evaluate, and improve the system incrementally.

A possible Agile development plan is:

Sprint 1 – Requirement Analysis and System Design: Identify stakeholders, define functional and non-functional requirements, analyze security risks, and design the system architecture.

Sprint 2 – User Interface and Authentication: Develop the web interface, login functionality, user roles, input forms, and basic dashboard.
Sprint 3 – URL and Email Analysis: Implement URL feature extraction, email text preprocessing, input validation, and rule-based detection.
Sprint 4 – Machine Learning Integration: Prepare suitable datasets, train and evaluate the classification model, and integrate the model with the detection engine.
Sprint 5 – Risk Scoring and Alerts: Implement risk categories, explanations, suspicious-content warnings, and available threat intelligence integration.
Sprint 6 – Incident Management and Reporting: Develop incident records, administrator functions, dashboards, audit logs, and security reports.
Sprint 7 – Testing and Evaluation: Conduct functional testing, security testing, usability testing, performance evaluation, and model validation. Correct identified defects and document the results.
Agile is appropriate because each sprint produces a testable component. The team can evaluate the model using previously unseen test data and improve it when weaknesses are identified. Feedback from users and cybersecurity analysts can also help reduce confusing warnings and improve the overall experience.
After deployment, the team should monitor detection quality, review new threat patterns, update datasets responsibly, and release improved model versions through controlled testing. This approach ensures that security and quality remain continuous activities rather than final-stage tasks.
Discuss Expected Outcomes and Limitations
The proposed system is expected to improve the identification and monitoring of phishing attempts by combining machine learning with established security checks. It provides users with risk assessments and explanations while giving administrators a centralized platform for reviewing detected threats.
Expected outcomes include:

- **Improved detection of phishing URLs and suspicious email content.**
- **Faster analysis of submitted content through automated feature extraction and classification.**
- **Clearer security warnings supported by explanations of relevant risk indicators.**
- **Centralized monitoring of detection results and reported incidents.**
- **Better visibility into phishing trends through dashboards and reports.**
- **Improved security awareness among users through understandable warnings.**
- **Better support for incident investigation and organizational security decisions.**

The effectiveness of the proposed solution should be measured through experiments rather than assumed. Useful metrics include precision, recall, F1-score, false-positive rate, false-negative rate, and average analysis time. Testing should use representative data that is separate from the training data to provide a more reliable estimate of performance.

Risks and Limitations

- **No detection system can guarantee the identification of every phishing attack.**
- **Machine learning models may incorrectly classify legitimate content or miss new attack patterns.**

- **Model performance depends on the quality, diversity, and freshness of training data.**
- **Attackers may deliberately modify URLs and messages to evade detection.**
- **External threat intelligence services may be unavailable, incomplete, or subject to usage limits.**
- **Collecting email content may introduce privacy and confidentiality concerns.**
- **Automated blocking can disrupt legitimate activity if a false positive occurs.**
- **The system requires ongoing maintenance, model evaluation, and security updates.**

These limitations can be reduced through regular testing, secure development practices, representative datasets, human review, clear risk explanations, and careful monitoring. The system should be considered an additional security layer rather than a complete replacement for email filtering, endpoint protection, user education, and other cybersecurity controls.

Future Enhancements

- **Develop a browser extension that warns users about potentially dangerous websites.**
- **Integrate the platform with organizational email gateways for controlled phishing detection.**
- **Incorporate additional threat intelligence sources and domain reputation checks.**
- **Develop explainable AI features that provide more detailed reasons for suspicious classifications.**
- **Use feedback from verified incidents to improve future model versions.**
- **Add automated model monitoring to identify changes in detection performance.**

Conclusion

The AI-Powered Phishing Detection and Cyber Threat Prevention System addresses the growing challenge of identifying fraudulent emails, suspicious URLs, and social engineering attempts. Traditional detection techniques remain useful for recognizing known threats, but their dependence on established rules and reported indicators can limit their effectiveness against unfamiliar or modified phishing attacks.

The proposed system combines machine learning, URL feature extraction, email text analysis, rule-based verification, risk scoring, and centralized incident management to provide an additional layer of cybersecurity protection. Its modular architecture supports maintainability, scalability, testing, and future integration with existing security tools.

From a software engineering perspective, the project demonstrates the importance of requirements analysis, modular design, secure coding, systematic testing, and continuous maintenance. Agile methodology supports incremental development and allows the team to improve the system using feedback from users and security professionals.

The most important lesson is that phishing prevention requires a combination of intelligent detection, secure software engineering, user awareness, and continuous improvement. By providing timely warnings, understandable risk assessments, and centralized threat monitoring, the proposed system can help individuals and organizations make safer decisions and strengthen their overall cybersecurity posture.